

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

CODE :

```
GNU nano 8.7.1 exp4c1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter Linux command: ");
    scanf("%99s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        execlp(command, command, (char *)NULL);
    }
}
```

```

GNU nano 8.7.1 exp4c1.c

if (pid == 0)
{
    // Child process
    printf("\nChild Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID : %d\n", getppid());

    execlp(command, command, (char *)NULL);

    // Executes only if execlp fails
    perror("exec failed");
    exit(1);
}
else
{
    // Parent process
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}

```

OUTPUT :

```

keerthi29@LAPTOP-VF1OP55D:~$ nano exp4.c
keerthi29@LAPTOP-VF1OP55D:~$ gcc exp4c1.c -oexp4c1
keerthi29@LAPTOP-VF1OP55D:~$ ./exp4c1
Enter Linux command: ls

Parent Process
Parent PID : 823
Child PID : 824

Child Process
Child PID : 824
Parent PID : 823
OSSP      exp2      file1.txt    program11   program6    q3_process_states  raed.c
OS_Practicals exp2.c      file2.txt    program11.c program6.c    q3_process_states.c read
command_executor exp3      input.txt    program4     program7     q4_wait            system
command_executor.c exp3.c      output.txt    program4.c   program7.c    q4_wait.c          system.c
expl      exp4c1     program10    program5     q2_file_copy q4_waitpid         write
expl.c    exp4c1.c   program10.c  program5.c   q2_file_copy.c q4_waitpid.c       write.c
Child process completed.
keerthi29@LAPTOP-VF1OP55D:~$

```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

CODE:

```

GNU nano 8.7.1 exp4c2.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0)
    {
        perror("Error creating destination file");
        close(source);
        return 1;
    }

```

```

GNU nano 8.7.1 exp4c2.c *
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead < 0)
    {
        perror("Error reading file");
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}

```

OUTPUT :

```

keerthi29@LAPTOP-VF10P55D:~$ nano source.txt
keerthi29@LAPTOP-VF10P55D:~$ ls
OSSP          exp2          exp4c2        input.txt     program4       program7       q4_wait       source.txt
OS_Practicals exp2.c        exp4c2.c     output.txt    program4.c     program7.c     q4_wait.c    system
command_executor exp3          exp5c1       program10.txt program5        q2_file_copy   q4_waitpid.c system.c
command_executor.c exp3.c       exp5c1.c     program10.c   program5.c     q2_file_copy.c q4_waitpid.c write
exp1          exp4c1       file1.txt    program11     program6       q3_process_states raed.c       write.c
exp1.c        exp4c1.c     file2.txt    program11.c   program6.c     q3_process_states.c read
keerthi29@LAPTOP-VF10P55D:~$ ./exp4c2
File copied successfully.

```

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

CODE:

```
GNU nano 8.7.1 exp4c3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- Child Process ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        printf("Child is running...\n");
    }
}
```

```
GNU nano 8.7.1 exp4c3.c *
    printf("\n--- Child Process ---\n");
    printf("Child PID : %d\n", getpid());
    printf("Child PPID: %d\n", getppid());

    printf("Child is running...\n");

    sleep(5);

    printf("Child is terminating.\n");
    exit(0);
}
else
{
    // Parent process
    printf("\n--- Parent Process ---\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    printf("Parent is waiting for child...\n");

    wait(NULL);

    printf("Child has terminated.\n");
    printf("Parent process terminating.\n");
}

return 0;
}
```

OUTPUT:

```
keerthi29@LAPTOP-VF10P55D:~$ nano exp4c3.c
keerthi29@LAPTOP-VF10P55D:~$ gcc exp4c3.c -oexp4c3
keerthi29@LAPTOP-VF10P55D:~$ ./exp4c3
Parent process started.
Parent PID: 1217
Parent PPID: 866

--- Parent Process ---
Parent PID : 1217
Child PID : 1218
Parent is waiting for child...

--- Child Process ---
Child PID : 1218
Child PPID: 1217
Child is running...
Child is terminating.
Child has terminated.
Parent process terminating.
```

-
4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:WAIT

```
GNU nano 8.7.1 exp4wait.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;
    pid_t pid;

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed.\n", i + 1);
            exit(0);
        }
    }

    // Parent waits for any child
    for (i = 0; i < 3; i++)
    {
        pid_t finished = wait(NULL);

        printf("Parent: Child with PID %d completed.\n", finished);
    }

    printf("All children completed.\n");

    return 0;
}
```

OUTPUT:

```

keerthi29@LAPTOP-VF10P55D:~$ nano exp4wait.c
keerthi29@LAPTOP-VF10P55D:~$ gcc exp4wait.c -oexp4wait
keerthi29@LAPTOP-VF10P55D:~$ ./exp4wait
Child 1: PID = 1285, PPID = 1284
Child 2: PID = 1286, PPID = 1284
Child 3: PID = 1287, PPID = 1284
Child 1 completed.
Parent: Child with PID 1285 completed.
Child 2 completed.
Parent: Child with PID 1286 completed.
Child 3 completed.
Parent: Child with PID 1287 completed.
All children completed.

```

CODE : WAITPID

```

GNU nano 8.7.1 exp4waitpid.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t children[3];
    int i;

    for (i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (children[i] == 0)
        {
            printf("Child %d: PID = %d, PPID = %d\n",
                   i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed.\n", i + 1);
            exit(0);
        }
    }
}

```

```
        return 1;
    }

    if (children[i] == 0)
    {
        printf("Child %d: PID = %d, PPID = %d\n",
               i + 1, getpid(), getppid());

        sleep(i + 1);

        printf("Child %d completed.\n", i + 1);
        exit(0);
    }
}

// Parent waits for each specific child
for (i = 0; i < 3; i++)
{
    waitpid(children[i], NULL, 0);

    printf("Parent: Child %d with PID %d completed.\n",
           i + 1, children[i]);
}

printf("All children completed.\n");

return 0;
```

OUTPUT :

```
keerthi29@LAPTOP-VF10P55D:~$ nano exp4waitpid.c
keerthi29@LAPTOP-VF10P55D:~$ gcc exp4waitpid.c -oexp4waitpid
keerthi29@LAPTOP-VF10P55D:~$ ./exp4waitpid
Child 1: PID = 1325, PPID = 1324
Child 2: PID = 1326, PPID = 1324
Child 3: PID = 1327, PPID = 1324
Child 1 completed.
Parent: Child 1 with PID 1325 completed.
Child 2 completed.
Parent: Child 2 with PID 1326 completed.
Child 3 completed.
Parent: Child 3 with PID 1327 completed.
All children completed.
```
